

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA0304

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

***CO2:** Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr DUMPALA PRASANTH

QUESTIONS: 10

Total Marks: 50

Annexure A **Application - Based Questions**

Duration: 2Hrs

1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality. **(5 Marks)**
2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations. **(5 Marks)**
3. Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time. **(5 Marks)**
4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance. **(5 Marks)**
5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism. **(5 Marks)**

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes. **(5 Marks)**

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization. **(5 Marks)**

8. Design and develop a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency. **(5 Marks)**

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance. **(5 Marks)**

10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function. **(5 Marks)**

Code of Conduct:

*I _Y.Thanushma(192425355)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Y.Thanushma

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. RL Framework for Ride-Sharing Driver–Passenger Matching

Understanding of Problem

A ride-sharing company needs to efficiently match passengers with available drivers. The main objectives are to minimize passenger waiting time, reduce unnecessary driver travel, and improve service quality. Reinforcement Learning can learn the best matching policy through repeated interaction with the ride-sharing environment.



RL Framework

- Agent: Ride-sharing matching system
- Environment: Drivers, passengers, road network, and traffic conditions

State Space

The state represents the current condition of the system:

- Locations of available drivers
- Locations of waiting passengers
- Number of available drivers and passengers
- Passenger waiting time
- Traffic conditions
- Estimated travel time
- Driver availability

Action Space

The agent can:

1. Assign a particular driver to a passenger.
2. Keep a driver available.
3. Reassign a driver to another passenger.

Reward Function

A suitable reward is:

$\text{Reward} = - \text{Passenger Waiting Time} - \text{Driver Travel Distance} - \text{Cancellation Penalty} + \text{Service Quality Reward}$

For example:

- Short waiting time → positive reward
- Successful matching → positive reward
- Long waiting time → negative reward
- Unnecessary driver movement → negative reward
- Passenger cancellation → large negative reward

Problem-Solving Approach

The agent observes the current state, selects a driver-passenger matching action, receives a reward, and updates its policy. By repeating this process, the system learns which assignments produce higher rewards.

Result

The learned policy can reduce passenger waiting time, improve driver utilization, reduce unnecessary travel, and increase overall service quality.

2. Exploration and Exploitation in E-Learning Recommendations

Understanding of Problem

An e-learning platform must recommend study materials that are useful for each student. However, recommending only familiar materials may prevent the system from discovering better resources. Therefore, RL uses exploration and exploitation to balance known and new recommendations.

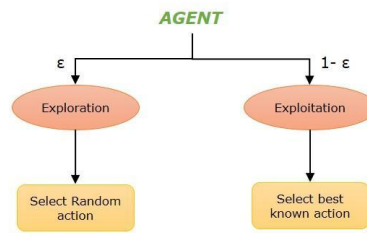
Exploration

- Exploration means trying new or less-known study materials.
- For example, if a student normally studies Python, the system may recommend a new Machine Learning video to determine whether it improves the student's learning.

Exploitation

- Exploitation means recommending materials that have already produced good results for the student.
- For example, if the student performs well after watching Python tutorials, the system continues recommending similar high-performing materials.

Epsilon Greedy Action Selection



RL Strategy

An ϵ -greedy strategy can be used:

- With probability ϵ , select a new/unexplored material.
- With probability $1 - \epsilon$, select the material with the highest estimated value.

Initially, the system performs more exploration. As it learns the student's preferences, it can increasingly exploit successful recommendations.

Reward Function

The reward can be based on:

Reward = Learning Improvement + Quiz Performance + Completion – Dropout

For example:

- High quiz score $\rightarrow$ positive reward
- Completing a lesson $\rightarrow$ positive reward
- Improved learning performance $\rightarrow$ positive reward
- Abandoning a lesson $\rightarrow$ negative reward

Result

The RL system learns personalized recommendations while still discovering new content, resulting in better student engagement, learning performance, and personalization.

3. RL Framework for Robotic Vacuum Cleaner

Understanding of Problem

A robotic vacuum cleaner must clean maximum area while avoiding obstacles and minimizing battery consumption. Reinforcement Learning allows the robot to learn an efficient cleaning policy through interaction with its environment.

RL Framework

- Agent: Robotic vacuum cleaner
- Environment: Room containing dirt, walls, furniture, and charging station

State Space

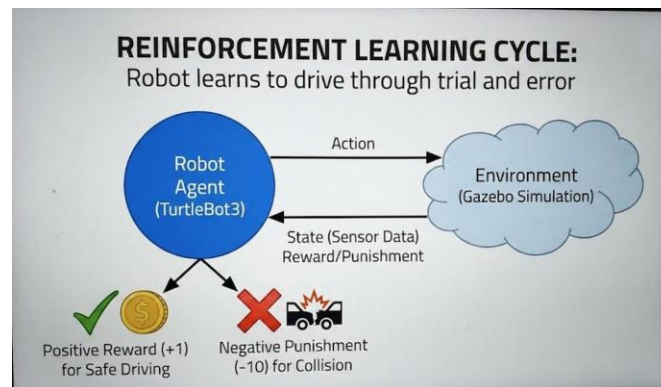
The state can contain:

- Robot's current position
- Dirt level
- Nearby obstacles
- Battery level
- Visited/unvisited areas
- Distance from charging station

Action Space

The robot can:

- Move forward
- Turn left
- Turn right
- Stop
- Return to charging station



Policy

A policy $\pi(a|s)$ determines which action the robot should take in a particular state.

For example:

- If dirt is nearby $\rightarrow$ move toward dirt.
- If an obstacle is detected $\rightarrow$ change direction.
- If the battery is low $\rightarrow$ return to charging station.
- If an area is already cleaned $\rightarrow$ move toward an uncleaned area.

Value Function

The value function $V(s)$ estimates the expected future reward from a particular state.

Initially, the robot may not know which states or routes are useful. Through repeated experience:

- Efficient cleaning states receive higher values.
- Collision states receive lower values.
- Energy-consuming states receive lower values.
- States leading to large cleaned areas receive higher values.

The value function therefore helps the robot select better actions and gradually improve its policy.

Result

Over time, the robot learns a policy that covers more area, avoids obstacles, reduces unnecessary movement, and saves battery energy.

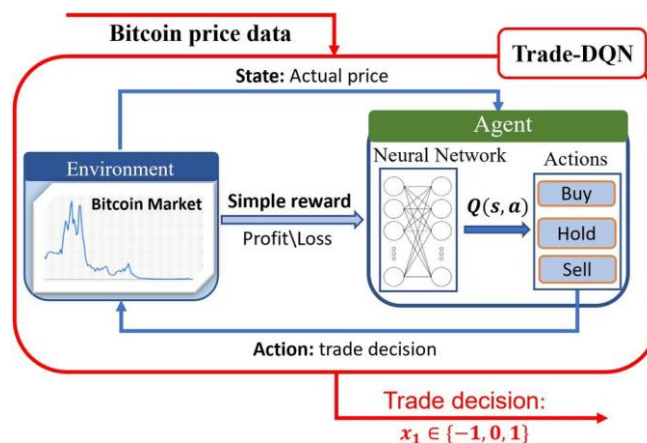
4. RL Cryptocurrency Trading System

Understanding of Problem

A cryptocurrency trading system can use RL to automatically decide whether to Buy, Sell, or Hold. The major challenge is designing a reward function that encourages profitability without encouraging excessive risk.

RL Framework

- Agent: Trading agent
- Environment: Cryptocurrency market



State Space

The state may include:

- Current cryptocurrency price
- Historical price changes
- Trading volume
- Portfolio balance
- Current holdings

- Technical indicators
- Market volatility

Action Space

The agent has three main actions:

Buy → Sell → Hold

Reward Function

A basic reward can be:

Reward = Change in Portfolio Value

A better reward should also consider trading costs and risk:

Reward = Profit – Transaction Cost – Risk Penalty

Challenges in Reward Design

1. Short-term vs long-term profit: A reward based only on immediate profit may produce poor long-term decisions.
2. Market volatility: Sudden price changes can produce unstable rewards.
3. Transaction costs: Ignoring transaction fees may encourage excessive trading.
4. Risk: The agent may take highly risky positions to obtain larger rewards.
5. Reward hacking: The agent may find actions that maximize the numerical reward without achieving the intended trading objective.

Effect of Poor Reward Design

If the reward only considers short-term profit, the agent may repeatedly trade or take excessive risks. It may obtain high rewards temporarily but suffer large losses during market changes.

Result

A carefully designed reward function helps the RL agent learn a profitable, risk-aware, and stable trading strategy.

5. RL-Based Smart Grid Electricity Distribution

Understanding of Problem

A smart grid must continuously balance electricity generation and demand while reducing energy cost, power losses, and peak loads. RL can learn an optimal energy distribution policy by observing grid conditions and receiving rewards.

RL Framework

- Agent: Smart grid controller

- Environment: Power generation units, consumers, batteries, renewable sources, and electricity network

State Space

The state can include:

- Current electricity demand
- Available power generation
- Battery charge level
- Solar and wind generation
- Electricity price
- Grid load
- Peak demand
- Energy storage status

Action Space

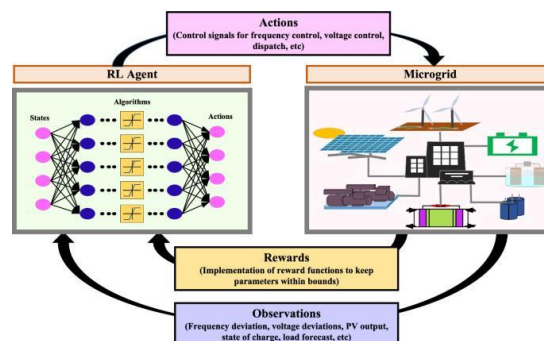
The agent can:

1. Increase or decrease electricity distribution.
2. Charge the battery.
3. Discharge the battery.
4. Use available renewable energy.
5. Shift flexible loads.
6. Control energy storage according to demand.

Reward Mechanism

The reward should encourage efficient and reliable energy management:

$$\text{Reward} = - \text{Energy Cost} - \text{Power Loss} - \text{Peak Demand} + \text{Renewable Energy Usage}$$



Therefore:

- Lower electricity cost → positive reward
- Lower transmission loss → positive reward
- Renewable energy usage → positive reward

- Reduced peak demand → positive reward
- Grid overload → negative reward
- Wasted energy → negative reward

Problem-Solving Approach

The agent follows these steps:

Observe Grid State → Select Action → Distribute/Store Energy → Calculate Reward → Update Policy → Repeat

Through repeated interaction, the agent learns which actions provide the highest long-term reward.

Result

The RL-based smart grid can reduce electricity costs, minimize power losses, reduce peak demand, improve renewable-energy utilization, and maintain grid stability.

6. RL Framework for Healthcare Medication Dosage

Understanding of Problem

A healthcare monitoring system can use RL to adapt medication dosage based on a patient's monitored condition. The objective is to maintain the patient within a safe target range while minimizing adverse effects.

Note: In a real healthcare system, dosage decisions must be supervised and constrained by clinicians; RL should not independently prescribe medication.

RL Framework

- Agent: Healthcare decision-support system
- Environment: Patient's monitored health condition

State Space

The state may include:

- Blood pressure
- Heart rate
- Blood glucose level
- Patient symptoms
- Previous dosage
- Medication response
- Relevant laboratory measurements

Action Space

The system can recommend:

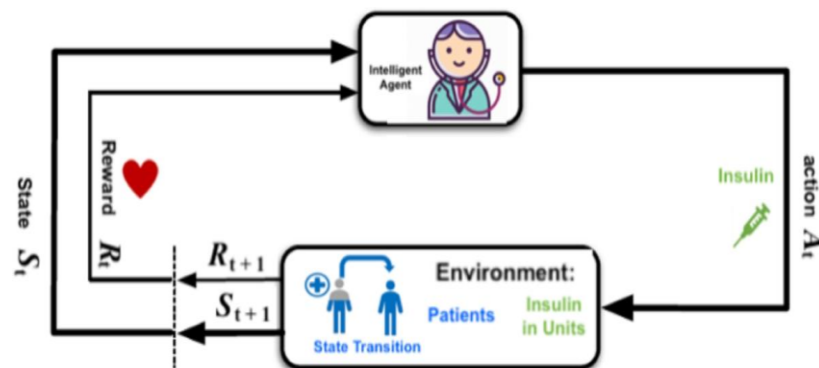
- Increase dosage
- Decrease dosage
- Maintain dosage
- Change timing, where clinically permitted

Reward Function

A safe reward can be designed as:

Reward = $-$ Deviation from Target $-$ Side-Effect Penalty $-$ Unsafe-Dose Penalty

- Health condition near target $\rightarrow$ positive reward
- Improvement in condition $\rightarrow$ positive reward
- Excessive deviation $\rightarrow$ negative reward
- Side effects $\rightarrow$ strong negative reward
- Unsafe dosage $\rightarrow$ strong negative reward



Learning and Improvement

The RL agent observes the patient's state, selects a clinically permitted action, receives feedback, and updates its policy. Over time, it can learn which actions are associated with better outcomes.

Result

The framework aims to provide personalized, safe, and efficient medication management under appropriate clinical supervision.

7. RL-Based Drone Delivery Navigation

Understanding of Problem

A drone delivery system must select efficient routes while avoiding obstacles, conserving battery, and reaching the destination safely. RL can learn navigation policies through repeated interaction with the environment.

RL Framework

- Agent: Delivery drone
- Environment: Airspace, buildings, obstacles, weather, and delivery locations

State Space

The state may include:

- Current drone position
- Destination
- Battery level
- Nearby obstacles
- Wind/weather conditions
- Current route
- Remaining distance

Action Space

The drone can:

- Move forward
- Move backward
- Move left/right
- Move upward/downward
- Change direction or route

Exploration

During exploration, the drone tries alternative routes that it has not used frequently.

Example:

Route A: Known and safe

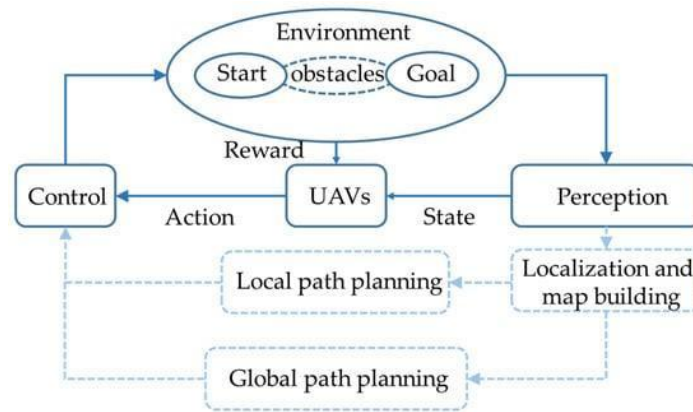
Route B: New but potentially shorter

Trying Route B helps the agent discover whether it provides better performance.

Exploitation

During exploitation, the drone selects the route that has already produced high rewards, such as a route with:

- Short travel time
- Low energy consumption
- Few obstacles



Balance

An ϵ -greedy strategy can be used:

- ϵ probability: Explore a new route
- $1 - \epsilon$ probability: Use the best-known route

Result

The drone gradually learns routes that reduce delivery time and battery consumption while maintaining safe navigation.

8. RL Framework for Manufacturing Robot Assembly

Understanding of Problem

A manufacturing robot performs repetitive assembly operations. RL can help it learn an efficient sequence of movements while reducing assembly time, errors, and unnecessary movement.

RL Framework

- Agent: Manufacturing robot
- Environment: Assembly workstation, components, tools, and machines

State Space

The state may include:

- Robot position
- Component position
- Assembly stage
- Tool status
- Remaining components

- Previous assembly action
- Error/failure status

Action Space

The robot can:

- Move to a component
- Pick a component
- Place a component
- Rotate a component
- Select a tool
- Move to the next assembly position

Policy

A policy $\pi(a|s)$ determines the best action for each assembly state.

For example:

If component is available $\rightarrow$ pick it $\rightarrow$ move to assembly position $\rightarrow$ place it $\rightarrow$ continue to next component.

The robot gradually learns a policy that minimizes unnecessary movements and assembly errors.

Value Function

The value function $V(s)$ estimates the expected future reward from a particular state.

For example:

- Correct assembly state $\rightarrow$ high value
- Fast successful assembly $\rightarrow$ high value
- Assembly error $\rightarrow$ low value
- Unnecessary movement $\rightarrow$ lower value

By updating the value function through experience, the robot learns which states and actions lead to efficient assembly.

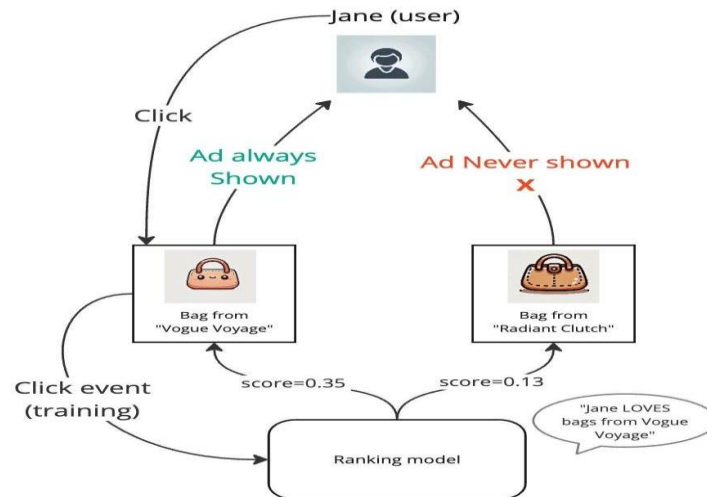
Result

The learned policy and value function help achieve faster assembly, fewer errors, reduced movement, and improved production efficiency.

9. RL-Based Online Advertising System

Understanding of Problem

An online advertising system can use RL to select advertisements that maximize click-through rate (CTR) while maintaining user experience and avoiding excessive or irrelevant advertisements.



RL Framework

- Agent: Advertisement recommendation system
- Environment: Users and online advertising platform

State Space

The state may include:

- User interaction history
- Current webpage/context
- Previous advertisements shown
- Time/session information
- Advertisement performance

Action Space

The agent can:

- Select Advertisement A
- Select Advertisement B
- Select Advertisement C
- Choose not to display an advertisement

Reward

A simple reward can be:

Reward = +1 for Click, 0 for No Click

However, reward design is challenging.

Reward Design Challenges

1. Clickbait: Optimizing only clicks may encourage misleading advertisements.
2. User experience: Too many advertisements may reduce user satisfaction.
3. Short-term vs long-term goals: A click does not necessarily mean a useful interaction.
4. Repeated advertisements: Showing the same advertisement repeatedly may annoy users.
5. Conversion quality: A click may not result in a purchase or meaningful engagement.

A better reward can combine:

$$\text{Reward} = \text{CTR} + \text{Conversion} - \text{User Experience Penalty} - \text{Repetition Penalty}$$

Impact on Performance

Poor reward design may increase CTR temporarily but reduce user satisfaction, conversions, and long-term engagement. A balanced reward helps the system optimize both advertising performance and user experience.

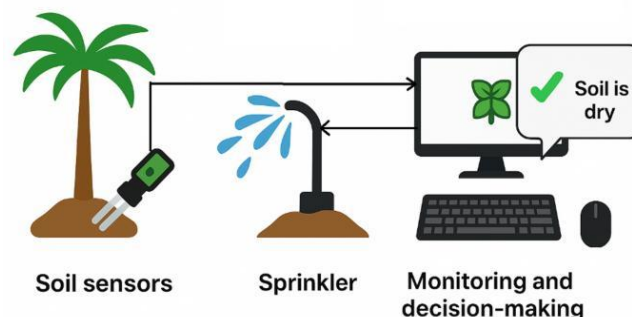
10. RL Framework for Smart Irrigation

Understanding of Problem

A smart irrigation system must provide enough water for healthy crop growth while minimizing water consumption. RL can learn when and how much water should be supplied based on environmental conditions.

RL Framework

- Agent: Smart irrigation controller
- Environment: Farm/crop field



State Space

The state can include:

- Soil moisture level
- Temperature

- Humidity
- Rainfall/forecast
- Crop growth stage
- Water tank level
- Previous irrigation amount

Action Space

The agent can:

- Start irrigation
- Stop irrigation
- Increase water quantity
- Decrease water quantity
- Select irrigation duration

Reward Function

A suitable reward is:

$\text{Reward} = \text{Crop Health} - \text{Water Consumption} - \text{Over-Irrigation Penalty}$

Therefore:

- Healthy crop → positive reward
- Water saving → positive reward
- Proper soil moisture → positive reward
- Excessive watering → negative reward
- Crop water stress → negative reward

Working

The system follows:

Sensors → Observe State → RL Agent → Irrigation Action → Measure Soil/Crop Condition
→ Reward → Policy Update

Through repeated learning, the agent discovers the irrigation strategy that provides sufficient water without unnecessary consumption.

Result

The RL-based irrigation system can reduce water wastage, maintain suitable soil moisture, improve crop health, and increase irrigation efficiency.